

UML Diagram Concepts

(Illustrated by OOP using Python)

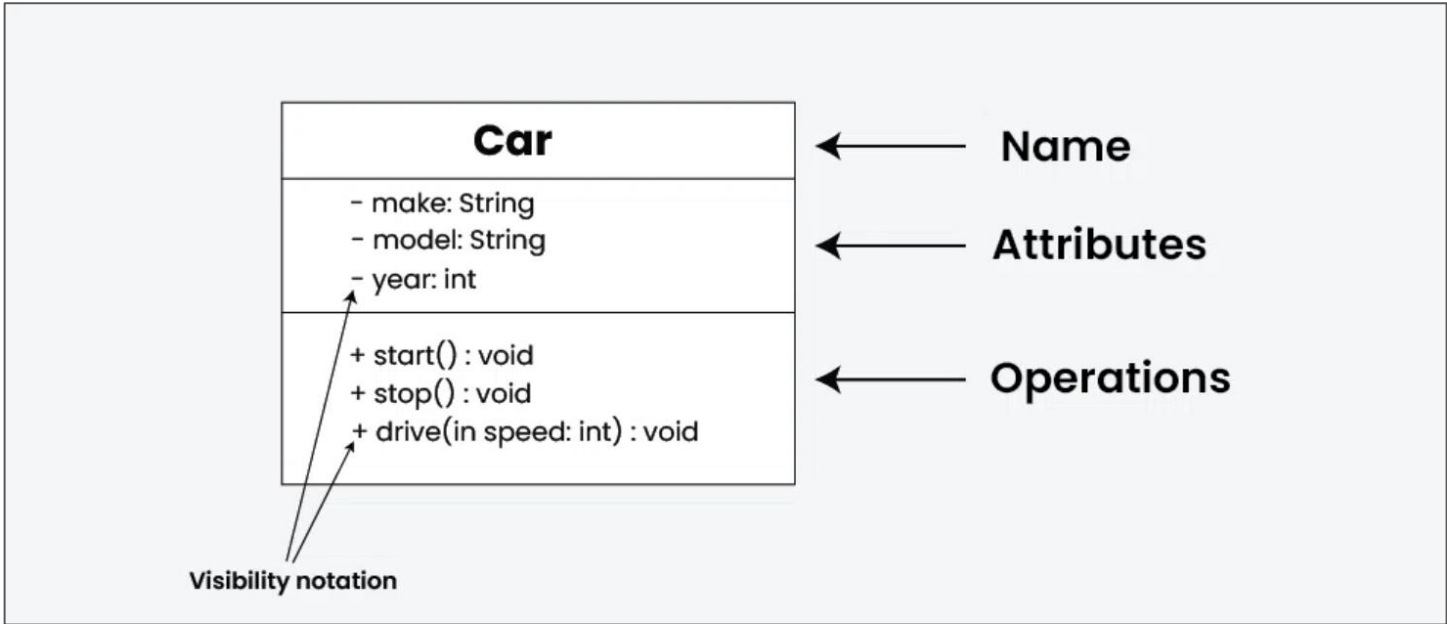
Introduction:

A UML class diagram shows the structure of a system by displaying its classes, their attributes, methods, and the relationships between them. It helps the team understand how the system is organized and how components interact.

- Represents classes, attributes, methods, and relationships
- Helps communicate and document the software structure
- Makes it easier for developers and designers to understand the system

UML Class Diagram Notations:

Classes are depicted as boxes, each containing three compartments for the class name, attributes/lists/dictionaries/variables/data members and methods/functions/operations.



In UML, a class is represented as a rectangle divided into three parts:

- **Top:** Class Name (In case of abstract class the class name is written in italic)
- **Middle:** Attributes (lists/dictionaries/variables/data members)
- **Bottom:** Operations (methods/functions)

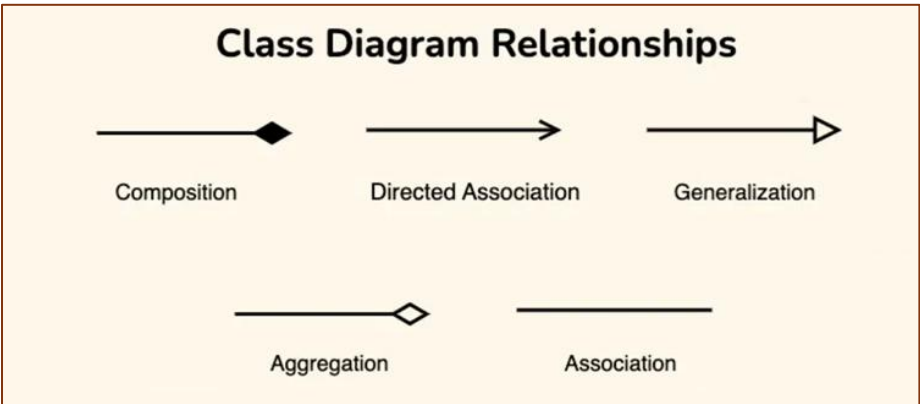
Visibility Markers

- + sign used for public variables
- - sign used for private variables
- # sign used for protected variables

Attributes/variables follow the format: visibility name : type

Methods/functions follow: visibility name(parameters) : returnType

Relationships:



In UML, **Association** is a broad "umbrella" term for any relationship where one object uses or interacts with another. It represents a "has-a" relationship where the objects have their own independent lifecycles.

In Python, there are technically **three** main scenarios of association, ranging from weak to strong links.

1. Simple Association (Peer-to-Peer)

This is a loose relationship. Two objects are aware of each other, but they exist independently. A common example is a **Doctor** and a **Patient**.

```
class Patient:
    def __init__(self, name):
        self.name = name

class Doctor:
    def __init__(self, name):
        self.name = name

    def check_up(self, patient):
        print(f"Doctor {self.name} is examining {patient.name}")

# Usage: They are created independently and linked during an action
doc = Doctor("Smith")
pat = Patient("Alice")
doc.check_up(pat)
```

1.1 Association Without Arrows Or Arrows at Both End (Undirected/Bi-directional)

In UML, a simple solid line often implies that the relationship is bi-directional, or that navigability is yet to be decided. In code, this means both classes hold a reference to each other.

```
class Person:
    def __init__(self, name):
        self.name = name
        self.employer = None # Reference to the other class

class Company:
    def __init__(self, company_name):
        self.company_name = company_name
        self.employees = [] # Reference to a list of Person objects

# Link them both ways
google = Company("Google")
mark = Person("Mark")

mark.employer = google
google.employees.append(mark)
```

Company
+String company_name +List<Person> employees
+init(company_name)

0..1 | works for / employs | *

Person
+String name +Company employer
+init(name)

The above code shows that a person class has 0 or 1 object of company class, while company class may have 1 or many object of person class, So the class diagram also shows same.

1.2 Association with Arrow at One End (Unidirectional)

This is the most common "Direct Association." The source class knows about the target, but the target has no knowledge of the source.

```
def __init__(self, name):
    self.name = name
    self.employer = None # Reference to the other class

class Company:
    def __init__(self, company_name):
        self.company_name = company_name

# Link them
google = Company("Google")
mark = Person("Mark")

mark.employer = google
```

Person
+String name +Company employer

works for > | 0..1

Company
+String company_name

2. Aggregation (Weak "Has-a")

This is a "part-of" relationship where the child can exist independently of the parent. If the parent is destroyed, the child remains.

- **Example:** A **Department** has **Professors**. If the department closes, the professors still exist.

```
class Professor:
    def __init__(self, name):
        self.name = name

class Department:
    def __init__(self, name, professors):
        self.professors = professors # Passed from outside

# Usage
p1 = Professor("Dr. Aris")
dept = Department("Computer Science", [p1])

del dept
print(p1.name) # p1 still exists even though dept is gone
```

Department
+String name +List<Professor> professors
+init(name, professors)

consists of >

Professor
+String name

This code demonstrates **Aggregation** because the Professor objects are created independently and passed into the Department. The lifecycle of the professor is not tied to the department; as your code shows, deleting the dept does not destroy p1. In UML, this is represented by a **hollow diamond** at the (container) class in our case Department contains object of professor so it is called container class.

3. Composition (Strong "Has-a")

This is a "death-grip" relationship. The child cannot exist without the parent. The parent is responsible for creating and destroying the child.

- **Example:** A **House** has **Rooms**. If the house is demolished, the rooms cease to exist.

```
class Room:
    def __init__(self, name):
        self.name = name

class House:
    def __init__(self):
        # The House creates its own Rooms; they don't exist outside it
        self.kitchen = Room("Kitchen")
        self.bedroom = Room("Bedroom")

# Usage
my_house = House()
del my_house
# Attempting to access my_house.kitchen now would result in a NameError
```

House
+Room kitchen +Room bedroom
+init()

contains >

Room
+String name

This code represents **Composition**, the strongest form of association. Because the Room objects are instantiated **inside** the House constructor, their existence is bound to the House. When my_house is deleted, the internal Room instances are also garbage collected (as they have no other references), fulfilling the "death-grip" relationship characteristic of composition. In UML, this is represented by a **filled (black) diamond** at the (container) class.

4. Generalization (Strong "is-a")

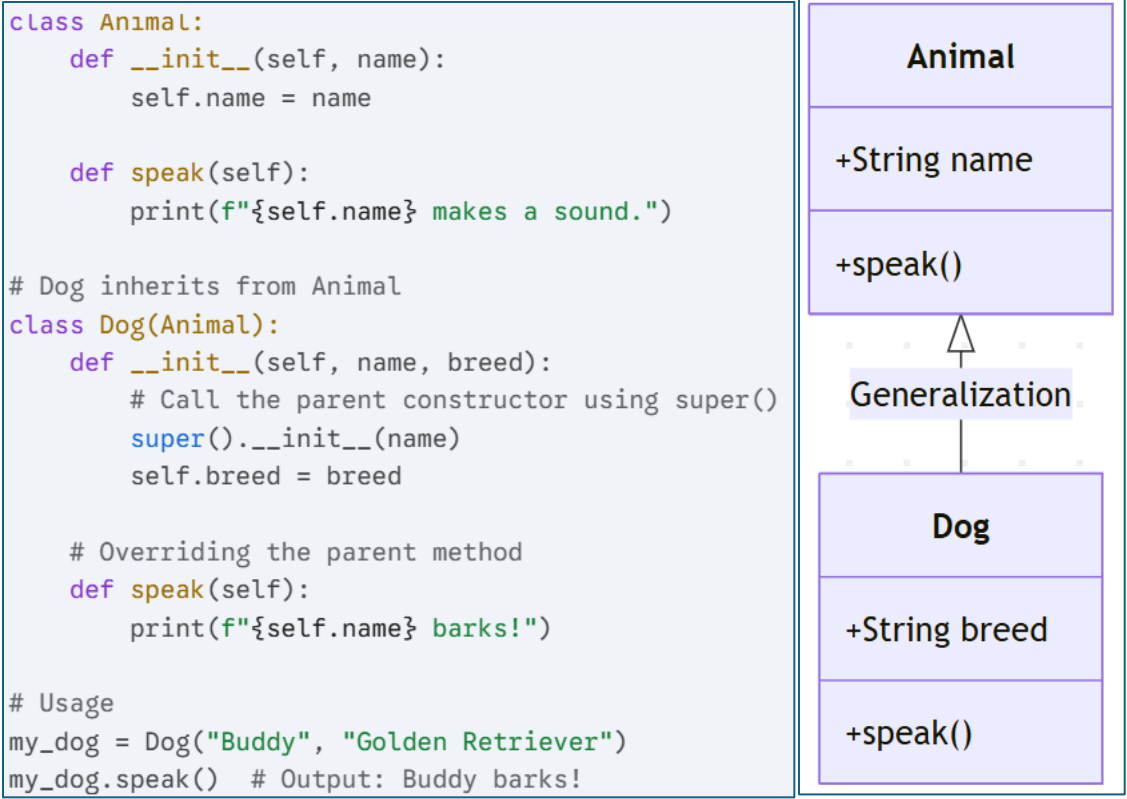
Inheritance (referred to as **Generalization** in UML) is a mechanism where a new class (the **Subclass** or Child) derives its attributes and behaviors from an existing class (the **Superclass** or Parent).

It represents an "is-a" relationship. For example, a Dog **is-a** Animal.

Core Characteristics

- **Code Reuse:** The subclass automatically gains all methods and attributes of the parent.
- **Method Overriding:** The subclass can provide a specific implementation of a method that is already defined in its parent.

- **The super() Function:** Used to call methods from the parent class, ensuring the parent's initialization logic is executed.



In a UML Class Diagram, Generalization is depicted by a **solid line with a hollow arrowhead** pointing from the Subclass to the Superclass.

Summary:

Type	Strength	Lifecycle	UML Notation
Association	Weakest	Independent	Solid line
Aggregation	Medium	Contained object survives Container Object	Hollow diamond
Composition	Strongest	Contained object dies with Container Object	Filled diamond
Generalization	Strong	Parent Child Class	Hollow Arrow

Code Scenario for Practice:

```
# 1. GENERALIZATION (Inheritance)
class Person:
    def __init__(self, name, email):
        self.name = name
        self.email = email

class Faculty(Person):
    def __init__(self, name, email, employee_id):
        super().__init__(name, email)
        self.employee_id = employee_id

class Student(Person):
    def __init__(self, name, email, student_id):
        super().__init__(name, email)
        self.student_id = student_id
        self.major_dept = None # To be linked via Association

# 2. THE COMPONENT (for Composition)
class Campus:
    def __init__(self, address):
        self.address = address

# 3. THE CONTAINER (Composition & Aggregation)
class Department:
    def __init__(self, dept_name):
        self.dept_name = dept_name
        self.staff = [] # Aggregation: List of Faculty

    def add_staff(self, faculty_member):
        self.staff.append(faculty_member)

class University:
    def __init__(self, name, location):
        self.name = name
        # COMPOSITION: The University creates its own Campus instance
        self.campus = Campus(location)
        self.departments = []

    def open_department(self, dept_name):
        new_dept = Department(dept_name)
        self.departments.append(new_dept)
        return new_dept

# --- Execution / Usage ---

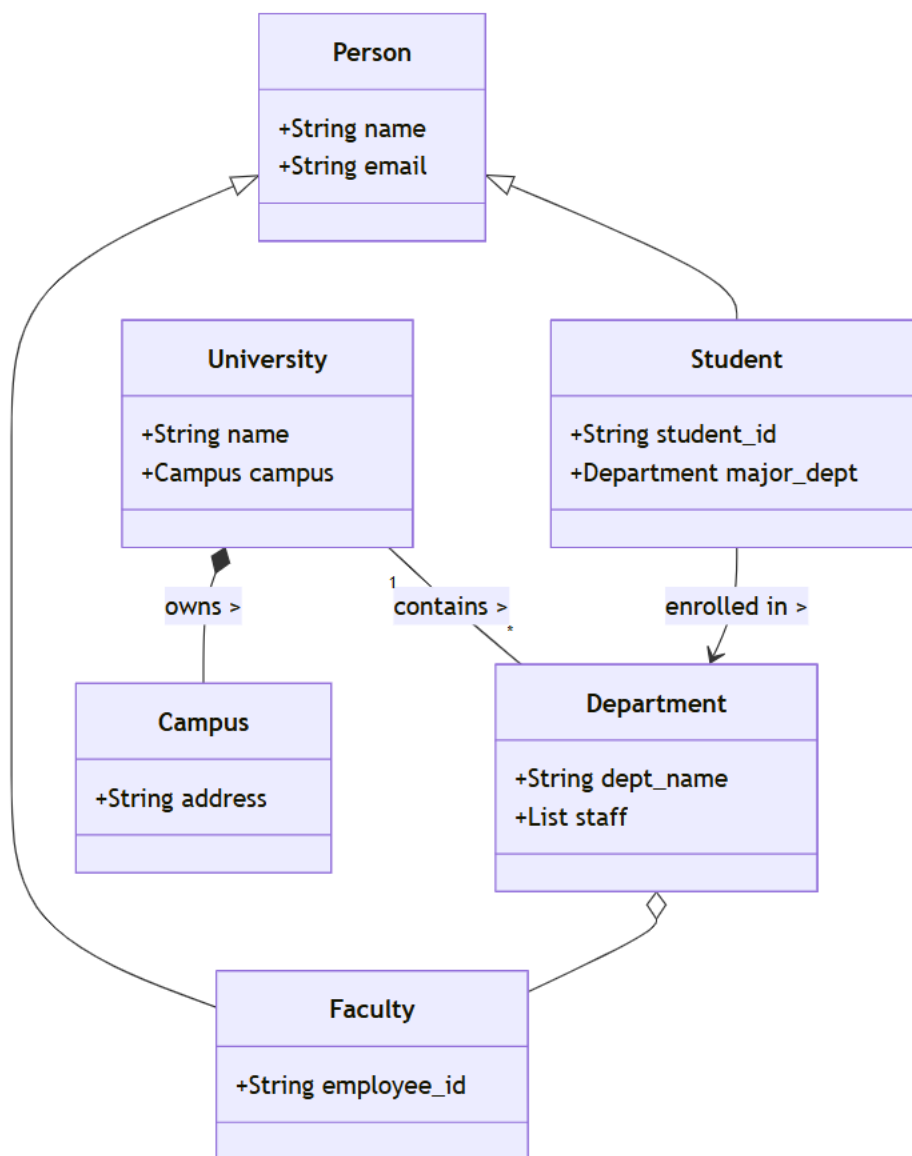
# Setup University and Campus (Composition)
my_uni = University("Tech State", "123 Innovation Drive")

# Setup Department
cs_dept = my_uni.open_department("Computer Science")

# Setup Faculty and link to Dept (Aggregation)
prof_smith = Faculty("Dr. Smith", "smith@tech.edu", "F990")
cs_dept.add_staff(prof_smith)

# Setup Student and link to Dept (Association)
student_mark = Student("Mark", "mark@tech.edu", "S101")
student_mark.major_dept = cs_dept # Mark is associated with CS

# Demonstration of Lifecycles
print(f"University {my_uni.name} is at {my_uni.campus.address}")
del my_uni
# Campus is now gone (Composition), but prof_smith and student_mark still
exist!
print(f"Professor {prof_smith.name} is still in memory.")
```



Description about code and class diagram:

- **Inheritance:** Faculty and Student inherit name from Person, showing shared identity.
- **Composition:** The Campus is instantiated *inside* University. If you del my_uni, the campus reference is lost.
- **Aggregation:** prof_smith is created *outside* the department. If cs_dept is deleted, prof_smith remains a valid object.
- **Association:** student_mark simply holds a reference to cs_dept. It's a "knows-about" relationship without ownership.